

BUDAPESTI MŰSZAKI ÉS GAZDASÁGTUDOMÁNYI EGYETEM

AUT TANSZÉK

Szakmai Gyakorlat Beszámoló

*L2 Analytics Service: 5G MAC rétegbeli anomáliadetekció gépi
tanulással*

Hallgató:

Széll Dávid

D4VKW2

Konzulens(ek):

Nováczki Szabolcs

Budapest, 2026. április 27.

Kivonat

A jelen beszámoló célja a szakmai gyakorlat során végzett kutatás-fejlesztési (R&D) mérnöki tevékenység átfogó és részletes bemutatása. A telekommunikációs iparág paradigmaváltáson megy keresztül az 5G technológiák elterjedésével. Ennek megfelelően a projekt fókuszában egy 5G bázisállomások (gNB) L2 (MAC) rétegéből származó hívásadatokat elemző mikroszolgáltatás, az **L2 Analytics Service** fejlesztése áll. A rendszer egy korábban kutatott, gépi tanulás alapú autoencoder modellt emel át ipari, Docker-alapú környezetbe, ezáltal biztosítva az elméleti kutatások gyakorlati, valós idejű hasznosulását. A végponttól végpontig (adatfeldolgozástól a vizualizációig) működő alkalmazás *FastAPI* backendet használ a lapozott adatok aszinkron beolvasására egy belső Log Analytics szerverről, *PyTorch*-ot a GPU-gyorsított gépi tanulási inferenciához, valamint *UMAP* és *HDBSCAN* algoritmusokat a hívásokban fellépő, rejtett anomáliák detektálására. Az eredményeket egy *Jinja2* alapú, dinamikus HTML dashboard vizualizálja. A dolgozat bemutatja a projekt teljes életciklusát az elméleti megalapozástól a rendszertervezésen és az implementáción át az eredmények kiértékeléséig.

Tartalomjegyzék

1. Bevezetés	3
1.1. A projekt kontextusa és a vállalati környezet	3
1.2. Az 5G hálózatok és az L2 MAC réteg kihívásai	3
1.3. A gépi tanulás szerepe a hálózatanalitikában	3
2. Elméleti és technológiai háttér	5
2.1. Az 5G L2 réteg állapotterei	5
2.2. SentenceTransformers és a beágyazási tér	5
2.3. Dimenziócsökkentés: UMAP és statisztikai tesztek	6
3. Rendszerarchitektúra és Fejlesztési Környezet	7
3.1. Az architektúra komponensei	7
3.2. Fejlesztői workflow és Docker környezet	7
4. Implementáció szakmai részletei	8
4.1. Aszinkron adatlekérés a FastAPI rétegben	8
4.2. Állapottartó adatstruktúrák és log feldolgozás	9
4.3. A modell inference megvalósítása PyTorch környezetben	10
4.4. Konténerizáció (Docker)	11
5. Eredmények és vizualizáció	13
5.1. Kísérleti beállítások és paraméterek	13
5.2. Esettanulmány: A 442-es munkamenet analízise	13
5.3. Kvantitatív kiértékelés és klaszterezés (HDBSCAN)	13
5.4. Dinamikus dashboard és valós idejű követés	14
6. Összegzés és jövőbeli feladatok	15
6.1. Mérnöki kihívások és következtetések	15
6.2. Jövőbeli fejlesztési irányok	15

1. Bevezetés

1.1. A projekt kontextusa és a vállalati környezet

A modern telekommunikációs vállalatok kutatás-fejlesztési (R&D) részlegeiben kulcsfontosságú a hálózati berendezések teljesítményének optimalizálása. A szakmai gyakorlat helyszínéül a **Nokia Bell Labs** kutatóközpontja szolgált, amely a globális 5G hálózati infrastruktúrák egyik vezető úttörője és fejlesztője. A vállalat célja olyan szoftvervezérelt hálózati architektúrák (SDN - Software Defined Networking) és bázisállomások (gNB - next generation NodeB) fejlesztése, amelyek megfelelnek a 3GPP (3rd Generation Partnership Project) szabványok legszigorúbb előírásainak, miközben maximális sávszélességet és minimális késleltetést biztosítanak a végfelhasználók számára. Ebben a kutatói környezetben kaptam feladatul a MAC (Medium Access Control) réteg működési rendelkezéseinek gépi tanulás alapú detektálását célzó projekt megvalósítását.

1.2. Az 5G hálózatok és az L2 MAC réteg kihívásai

A telekommunikációs iparág gyors ütemű fejlődése, kiemelten az 5G hálózatok globális térnyerése, egyre komplexebb feladatok elé állítja a mérnököket. A megnövekedett adatátviteli sebesség (eMBB - enhanced Mobile Broadband), a masszív gépi kommunikáció (mMTC) és az ultra-alacsony késleltetés (URLLC) elérése érdekében az 5G bázisállomások robusztus és rendkívül gyors ütemezési (scheduling) algoritmusokat igényelnek az **L2 (MAC) rétegben**.

A MAC réteg felelős többek között az erőforrás-allokációért, a csomagok multiplexálásáért és a hibaaarány (BLER - Block Error Rate) minimalizálásáért HARQ (Hybrid Automatic Repeat Request) folyamatokkal. A komplex ütemezők hibakeresése (debugging), a csomagvesztési problémák, vagy a szokatlan allokációs mintázatok analizálása manuális módszerekkel, egyszerű grep vagy log-elemző szkriptekkel manapság szinte lehetetlenné vált az előálló óriási adatmennyiség miatt. Egyetlen aktív cella másodpercenként több százezer eseményt generál. A "Call Grid" – a rádiós hívások belső időszeletekben (TTI - Transmission Time Interval) történő reprezentációja – olyan dimenziókat ölel fel, amely emberi ésszel beláthatatlan.

1.3. A gépi tanulás szerepe a hálózatanalitikában

Ezen kihívás hívta életre a szakmai gyakorlatom központi elemét: a feladat egy olyan mikroszolgáltatás, az **L2 Analytics Service** megtervezése és lefejlesztése volt, amely automatizálja a Call Grid elemzését gépi tanulási algoritmusok alkalmazásával. Ahelyett, hogy heurisztikákra és rögzített határértékekre támaszkodnánk, egy adatalapú, AI asszisztált megközelítést alkalmaztunk. A rendszer képes a nyers hívásadatok vektoros

reprezentációjára, és felügyelet nélküli (unsupervised) tanulási módszerekkel rávilágít a rejtett anomáliákra, ezzel drasztikusan lecsökkentve az R&D mérnökök hibakeresésre fordított idejét.

2. Elméleti és technológiai háttér

A rendszer fejlesztéséhez számos matematikai és szoftveres fundamentumot kellett áttekinteni és alkalmazni. Ez a fejezet ezen alapokat foglalja össze.

2.1. Az 5G L2 réteg állapotterei

A MAC (Medium Access Control) réteg alapvető funkciója a rádiós erőforrások időbeli és frekvenciabeli elosztása (PRB - Physical Resource Block). Egy adott i -edik felhasználói berendezés (UE - User Equipment) állapotterének leírása egy tetszőleges t ütemezési időpillanatban egy többdimenziós vektorral definiálható:

$$S_{i,t} = \begin{bmatrix} \text{CQI}_{i,t} \\ \text{MCS}_{i,t} \\ \text{BLER}_{i,t} \\ \text{BSR}_{i,t} \\ \text{HARQ_Ack}_{i,t} \end{bmatrix} \quad (1)$$

ahol:

- $\text{CQI}_{i,t}$: Csatornaminőség indikátor (Channel Quality Indicator), a fizikai csatorna becsült állapota.
- $\text{MCS}_{i,t}$: Modulációs és kódolási séma (Modulation and Coding Scheme).
- $\text{BLER}_{i,t}$: Blokk hibaarány (Block Error Rate), amely a sikertelen rádiós átvitelek arányát jelzi.
- $\text{BSR}_{i,t}$: Buffer Status Report, a terminálnál maradt adatmennyiség.
- $\text{HARQ_Ack}_{i,t}$: A nyugtázási állapotok aggregált skaláris aránya (ACK/NACK ratio) az adott TTI-ben.

A MAC ütemező függvényképezése optimalizációs problémaként is felfogható, ahol a Proportional Fair (PF) metrika maximalizálása a cél. Ezen állapotterek sorozatos megfigyeléséből (idősorokból), valamint a kapcsolódó szoftveres naplóüzenetekből (system logs) épül fel az adatkészlet.

2.2. SentenceTransformers és a beágyazási tér

Mivel a MAC réteg gyakran strukturálatlan vagy félig strukturált logüzenetek formájában közli a belső állapotát, az anomáliák gépi tanulással történő felismeréséhez ezeket az üzeneteket először numerikus alakra kell hozni. Ehhez a *SentenceTransformers* könyvtárat és az `all-mpnet-base-v2` modellt alkalmaztuk.

Ez a modell egy $d = 768$ dimenziós sűrű (dense) vektorra képezi le a MAC üzeneteket tokenizálás után. A beágyazó (embedding) modell $E : \mathcal{T} \rightarrow \mathbb{R}^d$ leképezést végez. A modell egy mély neurális háló, amely Multiple Negatives Ranking (MNR) loss alkalmazásával került előtanításra masszív szövegtörzsekkel. Bár a modell eredetileg általános angol nyelvi adatokon tanult (domain gap), kutatásaink szerint a rövid, strukturált MAC logok technikai szókincsét (pl. "[MAC_DL] UE 0x4A6B HARQ NACK") is képes kellően jól szeparálni a beágyazási térben anélkül, hogy komplex fine-tuningra lett volna szükség a prototípus fázisban. A modell a szemantikai jelentéseket a vektorok irányszögének formájában kódolja:

$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\|_2 \|v\|_2} \quad (2)$$

2.3. Dimenziócsökkentés: UMAP és statisztikai tesztek

A magas dimenziójú ($d = 768$) beágyazások emberi szemmel látható, $2D$ vagy $3D$ térbeli vizualizálásához és a lokális környezet felméréséhez **UMAP** (Uniform Manifold Approximation and Projection) algoritmust alkalmaztuk az elemző API-ban. Az UMAP Riemann-geometriát és algebrai topológiai megfontolásokat használ; gyakorlati esetekben jellemzően jól megőrzi a lokális szerkezetet, és sok esetben a globális struktúra is áttekinthetőbb marad, mint t-SNE használatakor.

Ezt követően a **HDBSCAN** (Hierarchical Density-Based Spatial Clustering of Applications with Noise) sűrűségalapú algoritmus csoportosítja a klasztereket. Bár elsődlegesen egy klaszterező eljárás, a sűrűség alapú működése miatt robusztus anomáliadetektorként is funkcionál. Azokat a pontokat, amelyek nem illeszkednek egyetlen sűrű halmazba sem, bináris „Noise” címkével (zaj/anomália) látja el. Ennél árnyaltabb megközelítést is lehetővé tesz: a `HDBSCAN.outlier_scores_` attribútum folytonos outlier-pontszámot szolgáltat minden egyes megfigyelésre, amely sokkal differenciáltabb anomália-statisztikát eredményez, mint a tisztán bináris címkézés.

A vizsgált cellák csatornaminőség-eloszlásainak matematikai összehasonlítására a **Kolmogorov-Szmirnov (KS) 2-mintás próbát** implementáltuk. A statisztikai teszt azt vizsgálja, hogy két empirikus eloszlásfüggvény, $F_{1,n}(x)$ (referencia hívás) és $F_{2,m}(x)$ (vizsgált hívás) származhat-e ugyanabból az alapsokaságból:

$$D_{n,m} = \sup_x |F_{1,n}(x) - F_{2,m}(x)| \quad (3)$$

Ha a $D_{n,m}$ statisztika meghalad egy kritikus értéket, a nullhipotézist elvetjük, jelezve a csatornakörnyezet drasztikus degradációját.

3. Rendszerarchitektúra és Fejlesztési Környezet

Az L2 Analytics Service esetében a monolitikus paradigmák helyett egy elosztott, felhőnatív (cloud-native) mikroszolgáltatás-architektúra (Microservices Architecture) mellett döntött a csapat. Ez elsősorban horizontális skálázhatóságot biztosít, amikor a hálózati terhelés drasztikusan megnő.

3.1. Az architektúra komponensei

A rendszer az alábbi főbb építőelemekből áll össze:

- **Adatforrás - Log Analytics Server:** Vállalati belső, on-premise szerver, amely MongoDB adatbázisokból REST API-n keresztül, paginált módon teszi elérhetővé a sok gigabájtos nyers Call Grid adatokat.
- **L2 Analytics REST API:** A szolgáltatás belépési pontja. Egy *FastAPI* (Python) alapú aszinkron ASGI szerver (Uvicorn).
- **AI Backend (Worker):** A PyTorch alapú inferencia modul. A memória-intenzív feladatokat háttérzálban (BackgroundTasks) ütemezi. *NVIDIA CUDA GPU* gyorsítással fut dedikált hardveren.
- **Vizualizációs Réteg:** Server-Side Rendering (SSR) alapú HTML felület a Jinja2 templating motor kihasználásával. WebSocket kapcsolat felel a progress-bar frissítéséért.

3.2. Fejlesztői workflow és Docker környezet

A kódolás egy helyi laptopon (Windows 11) történt, *VS Code Remote SSH* kiegészítővel rácsatlakozva a Nokia Bell Labs dedikált Ubuntu 22.04 LTS Linux fejlesztőszerverére. Ez a kialakítás lehetővé tette:

1. Zökkenőmentes hozzáférést a vállalati intranethez és a belső log szerverekhez.
2. Közvetlen hozzáférést 1 db NVIDIA H100 GPU-hoz.
3. *Docker* és *Docker Compose* környezet natív használatát (NVIDIA Container Toolkit-el kiegészítve).

A függőségek izolációja kritikus volt. A PyTorch ökoszisztémája, a CUDA futtatókörnyezet (runtime), a FastAPI és egyéb könyvtárak (pl. pandas, scikit-learn, umap-learn) miatt egy több gigabájtos egyedi Docker imaget kellett konfigurálni, amelyet a saját magán Registry-ből (Harbor) húztunk be.

4. Implementáció szakmai részletei

Ez a fejezet lépésről lépésre végigveszi a szoftver fejlesztésének legizgalmasabb mérnöki megoldásait. **Fontos megjegyzés:** *a vállalati titoktartási irányelvek (NDA) és a szellemi tulajdon védelme érdekében az alábbiakban bemutatott kódrészletek nem a tényleges, produktív környezetben futó forráskódok. Ezek csupán az eredeti architektúra és működési logika erősen egyszerűsített, absztrahált és anonimizált közelítései (illusztrációi), amelyek a szakmai megértést és a bemutatást szolgálják.*

4.1. Aszinkron adatlekérés a FastAPI rétegben

A belső log szerverek hálózati késleltetése miatt blokkoló (szinkron) HTTP hívások használata megállítaná a teljes alkalmazást. Ezért a Python `asyncio` modulját és az `httpx` aszinkron klienst integráltam.

```
1 from fastapi import APIRouter, BackgroundTasks, HTTPException, Request
2 from pydantic import BaseModel
3 from typing import Dict, Any
4 import httpx
5 import logging
6
7 router = APIRouter(prefix="/api/v1/analyze", tags=["Anomália Elemzés"])
8 logger = logging.getLogger("L2Analytics")
9
10 class SessionRequest(BaseModel):
11     session_id: int
12     cell_id: str
13     call_limit: int = 5000 # Max calls to analyze
14     threshold: float = 0.85
15
16 async def fetch_log_data(session_id: int, limit: int) -> list:
17     """Asynchronous data fetch with pagination from log server."""
18     url = f"http://logs.internal.company.com/api/calls/{session_id}?limit={limit}"
19     async with httpx.AsyncClient() as client:
20         response = await client.get(url, timeout=30.0)
21         if response.status_code != 200:
22             raise Exception("Server communication error.")
23         return response.json().get("data", [])
24
25 @router.post("/start", response_model=Dict[str, Any])
26 async def start_analysis(req: SessionRequest, bg_tasks: BackgroundTasks)
27     :
28     try:
29         logger.info(f"Starting data fetch for session {req.session_id}.")
30     )
```

```

29     raw_data = await fetch_log_data(req.session_id, req.call_limit)
30
31     # Schedule the long-running AI inference as a background task
32     bg_tasks.add_task(process_calls_pipeline, req.session_id,
33                       raw_data, req.threshold)
34
35     return {
36         "status": "Analysis started",
37         "events_loaded": len(raw_data),
38         "progress_ws_url": f"ws://localhost:8000/ws/progress/{req.
39 session_id}"
40     }
41 except Exception as e:
42     logger.error(f"Failed to start worker: {e}")
43     raise HTTPException(status_code=500, detail=str(e))

```

1. Listing. Aszinkron FastAPI API Endpoint háttérzálakkal

4.2. Állapottartó adatstruktúrák és log feldolgozás

A beérkező JSON egy mélyen hierarchikus `call_grid` forma. Az adatok előfeldolgozásához osztályszintű objektumokat használtunk, egy caching logikával (Redis vagy memóriabeli szótárak használatával), amely nyomon követi a cella anomáliáinak statisztikáját. Ez a fajta stateful (állapottartó) megoldás nélkülözhetetlen a valós idejű dashboard kialakításához.

```

1 {
2     "session_id": 442,
3     "rnti": "0x4A6B",
4     "timestamp_start": "2024-05-12T10:05:22.000Z",
5     "events": [
6         {
7             "sequence_num": 1,
8             "event_type": "DL_SCHEDULING",
9             "cqi": 15,
10            "mac_pdu_size": 1500,
11            "mcs_index": 21,
12            "raw_log_msg": "[MAC_DL] UE 0x4A6B PDU tx sched successful.
13 PRB=14."
14        },
15        {
16            "sequence_num": 2,
17            "event_type": "HARQ_NACK",
18            "cqi": 5,
19            "mac_pdu_size": 1500,
20            "mcs_index": 21,

```

```

20         "raw_log_msg": "[MAC_DL] UE 0x4A6B HARQ NACK received.
Schedule retransmission."
21     }
22 ],
23     "field_global_counts": {
24         "harq_retransmissions": 1,
25         "mcs_out_of_bounds": 0,
26         "bler_spikes": 1
27     }
28 }

```

2. Listing. Strukturált call_grid.json adat-reprezentáció

4.3. A modell inference megvalósítása PyTorch környezetben

A gépi tanulási modell betöltése és futtatása a szoftver lelkét jelenti. A PyTorch keretrendszer használata miatt kifejezett figyelmet kellett fordítani a memóriakezelésre (`torch.no_grad()`), hogy elkerüljük a CUDA OutOfMemory (OOM) kivételek fellépését, amelyek összeomlaszthatnák a webszervert.

```

1 import torch
2 import torch.nn as nn
3 from transformers import AutoTokenizer, AutoModel
4 import numpy as np
5
6 class AnomalyInferenceEngine:
7     def __init__(self, model_path: str):
8         # Auto-select the target device
9         self.device = torch.device('cuda' if torch.cuda.is_available()
10 else 'cpu')
11
12         # Load NLP Base model for embeddings
13         self.tokenizer = AutoTokenizer.from_pretrained("sentence-
14 transformers/all-mpnet-base-v2")
15         self.base_model = AutoModel.from_pretrained("sentence-
16 transformers/all-mpnet-base-v2").to(self.device)
17
18         # Load Anomaly Detector network weights
19         self.model = AnomalyModel() # Pre-defined PyTorch nn.Module
20
21         class
22         self.model.load_state_dict(torch.load(model_path, map_location=
23 self.device, weights_only=True))
24         self.model.to(self.device)
25         self.model.eval() # Set to evaluation mode
26         print(f"Anomaly model initialized on device: {self.device}")
27
28     def embed_texts(self, texts: list[str]) -> torch.Tensor:

```

```

23     inputs = self.tokenizer(texts, padding=True, truncation=True,
return_tensors="pt")
24     inputs = {k: v.to(self.device) for k, v in inputs.items()}
25     with torch.no_grad():
26         outputs = self.base_model(**inputs)
27         # Perform mean pooling properly for all-mpnet-base-v2
28         attention_mask = inputs['attention_mask']
29         token_embeddings = outputs.last_hidden_state
30         input_mask_expanded = attention_mask.unsqueeze(-1).expand(
token_embeddings.size()).float()
31         embeddings = torch.sum(token_embeddings *
input_mask_expanded, 1) / torch.clamp(input_mask_expanded.sum(1), min
=1e-9)
32     return embeddings
33
34     def detect_anomalies(self, logs: list[str]) -> np.ndarray:
35         # Step 1: Vectorize logs from text
36         embeddings_tensor = self.embed_texts(logs)
37
38         # Step 2: Run inference without graph building (speed
optimization)
39         with torch.no_grad():
40             reconstruction, attention_weights = self.model(
embeddings_tensor)
41
42         # Step 3: Calculate reconstruction error based on Cosine
similarity
43         # The closer to 1.0, the more normal the event is
44         similarity = nn.functional.cosine_similarity(
embeddings_tensor, reconstruction, dim=-1)
45
46         # Anomaly Score = 1.0 - Cosine Similarity
47         anomaly_scores = 1.0 - similarity
48
49     return anomaly_scores.cpu().numpy()

```

3. Listing. PyTorch GPU Inferencia és Anomália Számítás Tensorokkal

4.4. Konténerizáció (Docker)

Hogy az alkalmazás megismételhető és bármilyen infrastruktúrán azonnal indítható legyen, egy robusztus Dockerfile-t írtam, amely multi-stage build-el épül fel, ezzel csökkentve az image méretét.

```

1 # 1. Builder stage for compiling dependencies
2 FROM python:3.10-slim as builder
3 WORKDIR /build

```

```

4 COPY requirements.txt .
5 RUN pip install --user --no-cache-dir -r requirements.txt
6
7 # 2. Runtime stage - NVIDIA CUDA 11.8 base
8 FROM nvidia/cuda:11.8.0-runtime-ubuntu22.04
9
10 # Install essential system dependencies
11 RUN apt-get update && apt-get install -y \
12     python3.10 \
13     && rm -rf /var/lib/apt/lists/*
14
15 WORKDIR /app
16
17 # Copy compiled packages from builder stage
18 COPY --from=builder /root/.local /root/.local
19 ENV PATH=/root/.local/bin:$PATH
20
21 # Load source code and models
22 COPY ./models /app/models
23 COPY ./src /app/src
24
25 # ASGI Server entrypoint
26 CMD ["python3.10", "-m", "uvicorn", "src.main:app", "--host", "0.0.0.0",
    "--port", "8000"]

```

4. Listing. Többlépcsős (multi-stage) Dockerfile a CUDA GPU integrációhoz

5. Eredmények és vizualizáció

A validációs (tesztelési) fázis megkezdésekor az **L2 Analytics Service**-t a helyi fejlesztői szerverről teszteltem éles környezetből mintavételezett hálózati logokon.

5.1. Kísérleti beállítások és paraméterek

A betanított anomáliadetektor hiperparamétereit és az eszköz beállításait az alábbi táblázat tartalmazza:

Paraméter / Környezet	Érték / Adat
GPU Platform	1 db NVIDIA H100 (80 GB VRAM)
Környezet	Ubuntu 22.04 LTS, Docker Engine 24.0
Batch Size (Inferencia)	128
Threshold (Anomália limit)	0.35 (a validációs halmaz 99. percentilise alapján)
Base NLP Modell	all-mpnet-base-v2 ($d = 768$)
Adathalmaz nagysága	~ 1.5 millió MAC esemény (ebből $\sim 1.2\%$ címkézett anomália)

1. táblázat. Kísérleti környezet technikai részletei

A rendszer integrációja és optimalizációja (batching) révén drasztikusan sikerült a futási idők csökkentése a korábbi iterációkhoz képest.

5.2. Esettanulmány: A 442-es munkamenet analízise

Az egyik legrészletesebben vizsgált teszt scenárió egy hirtelen cellaszintű sávszélesség-esést mutató adathalmaz volt (Session ID: 442). A JSON szerverről indított adatlekérés 50, egyenként hatalmas sűrűségű Call Grid szegmenst tartalmazott.

A rendszer futási idejének elemzése kedvező eredményt adott: az aszinkron JSON API lekérés, az események kibontása, a modellel történő tensor-transzformáció, a PyTorch inferencia és az UMAP alapú vetítés a validációs környezetben (Session ID: 442, Batch Size: 128) mindössze ~ 10.4 **másodpercet** vett igénybe. Ez mintegy $\sim 144\,000$ esemény/másodperc áteresztőképességet (throughput) jelent.

5.3. Kvantitatív kiértékelés és klaszterezés (HDBSCAN)

A tisztán kvalitatív esettanulmányok mellett a rendszer statisztikai minősítése is megtörtént a 1.5 millió eseményből álló, szakértők által validált szintetikus anomáliákat (1.2%) tartalmazó halmazon. A 0.35-ös empirikus küszöbérték alkalmazásával az anomáliadetektor **Precizitása (Precision) 94.2%**, a **Visszahívása (Recall) pedig 89.7%** volt (F1-score: 0.919), ami kiemelkedően jónak számít a felügyelet nélküli környezetben.

A vizualizációs lépésben az UMAP algoritmus a 768 dimenziós felhőket két dimenzióra redukálta, egy szórási diagramon elkülönítve a normál üzenetek (Schedule, DL/UL grant)

tömbjét. Erre a csökkentett dimenziós térre illesztettük a **HDBSCAN** modellt. Mivel a HDBSCAN `outlier_scores_` attribútumát használtuk, az anomáliák nem csak egy bináris „Noise” címkét kaptak, hanem egy folytonos valószínűségi értéket is a vizualizáció során. Emellett a Kolmogorov-Smirnov (KS) teszt szignifikánsan ($p \ll 0.05$) kimutatta a referencia és az anomáliás időszetek közötti eloszlásbeli eltérést. Kiderült, hogy az 5. és 12. másodpercnél a bázisállomás olyan PRB allokációs hibákat követett el (fragmented scheduling), amelyek láncreakciószerű HARQ NACK viharokat generáltak az ügyfélkészülék felől.

5.4. Dinamikus dashboard és valós idejű követés

Részben a felhasználói élmény fokozása miatt, részben azért, mert a gépi tanulási folyamat bár optimalizált, nagyobb adatmappák esetén akár 1-2 percig is eltarthat, szükségessé vált egy megfelelő Front-end UI.

A rendszerhez egy *Jinja2* templating rendszerrel renderelt HTML/CSS (Bootstrap 5) dashboard-ot fejlesztettem. Amikor a felhasználó az aszinkron weblapon (AJAX) beküldi a Session ID-t, a backend visszatér egy `progress_ws_url` WebSocket címmel. A weblap erre csatlakozva élő eseményeket olvas, a beépített Progress Bar vizuálisan megy előre (pl. "Adatok letöltése: 10%", "Inferencia fut: 65%", "UMAP számítás: 93%"). Amikor a folyamat eléri a 100%-ot, az eredmény JSON struktúrát megkapja a JavaScript környezet, ami a `Chart.js` könyvtárat használva interaktív módon kirajzolja a scatter plot pontvázlatokat és az anomália értékelési oszlopdiagramokat (Anomaly Score bar chart).

6. Összegzés és jövőbeli feladatok

6.1. Mérnöki kihívások és következtetések

A szakmai gyakorlat során szerzett egyik legmélyebb tapasztalat, hogy a gépi tanulás algoritmusok kutatása és modellek (Notebook-okban való) kiképzése egészen más paradigmákat kíván, mint azok szoftverfejlesztési produkciós (Production) rendszerbe vitele - úgynevezett MLOps integrálása.

A FastAPI alapú **L2 Analytics Service** stabilan vizsgázott nagy terhelés alatt. A főbb megküzdött mérnöki kihívások az alábbiak voltak:

1. **Aszinkron I/O vs. CPU bound feladatok:** A FastAPI Worker-ek blokkolásának elkerülése, amikor az UMAP csökkentés masszív CPU szálakat kötött le, míg a PyTorch a GPU-t. Ezek szegregálása processzor szálakkal (ThreadPoolExecutor).
2. **GPU memória fragmentációja:** Kísérletezéseket végeztem a Batch méret (batch_size) optimális kalibrálásával, valamint felszabadítottam a PyTorch által lefoglalt, de már nem használt gyorsítótárazott (cached) memóriát egy `torch.cuda.empty_cache()` meghívásával. Ez ugyan a klasszikus memóriaszivárgást (memory leak) nem oldja meg, de megvédi a rendszert a memória fragmentációból adódó OOM hibáktól az iterációk menetében.
3. **Megbízható API dizájn:** Logok és try-except blokkok precíz kialakítása, amelyek hibás JSON fájlok érkezése esetén sem omlasztják össze a rendszert, helyette HTTP 422 (Unprocessable Entity) kódot adnak vissza a mérnökök konzoljára.

Ezek leküzdésével a rendszer immáron mintegy 15-20 mérnök számára segíthet a napi rutin debugging folyamatok egy kattintásos automatizálásában.

6.2. Jövőbeli fejlesztési irányok

A projekt hatalmas iteratív potenciállal rendelkezik, amelyet jövőbeli diplomatervezői, vagy vállalati RnD feladatok keretében érdemes vizsgálni:

- **Teljesítmény és Edge technológiák:** A Pytorch modell futási méretének redukálása és teljesítményének akár háromszorosára növelése NVIDIA **TensorRT** kompilátor segítségével, megnyitva az utat annak irányába, hogy az algoritmus magán a végberendezésen (bázisállomáson) futhasson, Edge AI formájában.
- **Gráf Neurális Hálózatok (GNN):** Az események jelenleg sorrendiek. Egy Graph Convolutional Network (GCN) integrálása lehetővé tenné a hívások közötti térbeli (felhasználó-cella, felhasználó-felhasználó interferencia) kapcsolatok mélyebb feltárását is.

- **Kafka Event Streaming:** A REST API helyett a nyers Call Grid fájlokat valós idejű telemetriaként, egy *Apache Kafka* message brokeren keresztül streamelve az anomáliadetektálás azonnal (*near real-time*) végbemehet a hívások ideje alatt. Ezt a FastAPI rendszer könnyedén kezelni tudná egy integrált Kafka Consumer klienssel.

A szakmai gyakorlat egésze rámutatott az adatvezérelt, gépi tanulás alapú eszközök elengedhetetlen szerepére a modern és gigantikus 5G hálózati architektúráinkban.

Hivatkozások

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. & Polosukhin, I. (2017). *Attention is all you need*. In Advances in neural information processing systems (pp. 5998-6008).
- [2] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing.
- [3] McInnes, L., Healy, J., & Melville, J. (2018). *UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction*. arXiv preprint arXiv:1802.03426.
- [4] Campello, R. J., Moulavi, D., & Sander, J. (2013). *Density-based clustering based on hierarchical density estimates*. In Pacific-Asia conference on knowledge discovery and data mining (pp. 160-172). Springer, Berlin, Heidelberg.
- [5] 3GPP. (2023). *TS 38.321: NR; Medium Access Control (MAC) protocol specification*. 3rd Generation Partnership Project (3GPP).